

Progettazione Concettuale

Il modello Entità/Associazioni (E/R)

Informatica — Classe Quinta, Istituto Tecnico

Indice degli argomenti

1. Progettazione concettuale e UML
2. Il modello Entità/Associazioni (E/R)
3. Modellare i dati: livelli concettuale/logico/fisico
4. Entità, dominio, attributi (semplici, composti, derivati)
5. Rappresentazione grafica degli attributi
6. Chiave primaria (PK) e chiave primaria parziale (PPK)
7. Entità forti e deboli
8. Associazioni: attributi e grado (binarie, ternarie, ricorsive)
9. Molteplicità e cardinalità, tipi di associazione
10. Regole di lettura e simboli grafici (Chen e crow's foot)
11. Esempio completo e importanza dell'analisi del problema

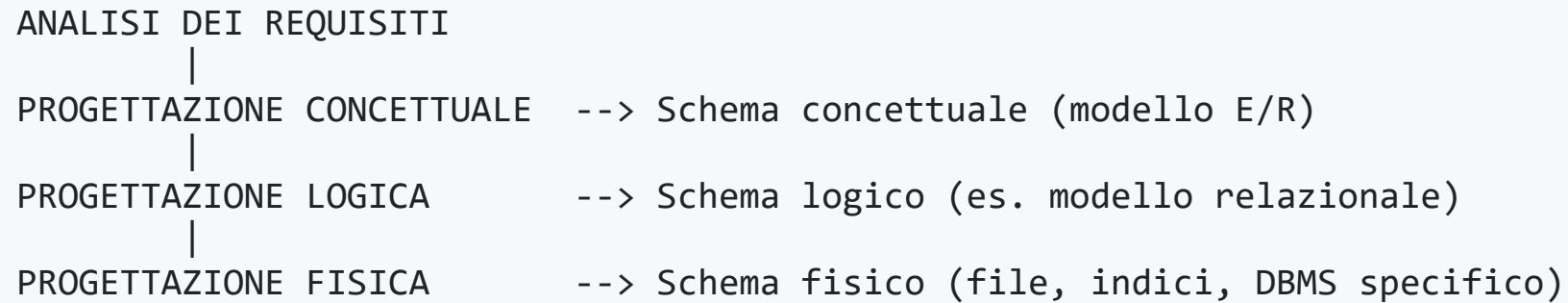
Cos'è la progettazione concettuale

La **progettazione concettuale** è la prima fase del processo di progettazione di una base di dati: consiste nel **rappresentare la realtà di interesse** in modo formale, **indipendente da qualsiasi tecnologia o DBMS**.

- Obiettivo: descrivere **cosa** deve contenere il database, non **come** verrà implementato
- Produce uno **schema concettuale**, comprensibile sia dai tecnici sia dagli esperti del dominio (committenti)
- Lo strumento standard per la progettazione concettuale è il **modello Entità/Associazioni (E/R)**

Esempio: prima di creare le tabelle SQL di un database scolastico, si analizza la realtà (studenti, classi, docenti, materie) e si rappresenta con uno schema concettuale, **senza** ancora pensare a tabelle o SQL.

Le fasi della progettazione di un database



Procedere per fasi separate consente di **non mescolare** le scelte concettuali (cosa rappresentare) con quelle implementative (come memorizzarlo), riducendo errori e facilitando la manutenzione.

Cos'è UML

UML (Unified Modeling Language) è un linguaggio di modellazione grafico, standardizzato, usato principalmente per la progettazione **object-oriented** del software.

- Fornisce diversi tipi di diagrammi: **diagrammi delle classi**, dei casi d'uso, di sequenza, di attività, ecc.
- Il **diagramma delle classi UML** può essere usato anche per la progettazione concettuale di basi di dati, in alternativa (o accanto) al modello E/R
- Concetti come **classe**, **attributo**, **associazione**, **molteplicità** in UML corrispondono concettualmente a entità, attributo, associazione e cardinalità nel modello E/R

Nota per l'esame: nei testi di seconda prova può essere richiesto sia il diagramma E/R sia, in alternativa, il diagramma delle classi UML: i concetti di base (associazioni, molteplicità) sono analoghi.

Il modello Entità/Associazioni (E/R)

Il modello **Entità/Associazioni** (Entity/Relationship, **E/R**), proposto da Peter Chen nel 1976, è lo strumento standard per la **progettazione concettuale** delle basi di dati.

Rappresenta la realtà attraverso tre costrutti fondamentali:

- **Entità**: oggetto (concreto o astratto) che ha un significato anche quando viene considerato in modo isolato ed è di interesse per la realtà che si vuole modellare (es. `Studente` , `Corso`)
- **Associazioni** (relationship): legami logici che stabilisce un'interazione tra due o più entità (es. `Studente frequenta Corso`)
- **Attributi**: proprietà che descrivono entità o associazioni (es. `Cognome` , `DataNascita`)

È detto anche **modello Entità/Relazioni**: attenzione a non confondere il termine "relazione" qui usato con la "relazione" del modello relazionale (che indica una tabella) — sono concetti diversi con lo stesso nome!

Modellare i dati: cosa significa

Modellare i dati significa costruire una **rappresentazione astratta e semplificata** della realtà di interesse, individuando gli oggetti rilevanti, le loro proprietà e le relazioni che li legano, in funzione degli scopi del sistema informativo.

- Non si rappresenta *tutta* la realtà, ma solo gli aspetti **rilevanti per l'applicazione**
- Il modello deve essere **corretto** (fedele alla realtà), **completo** (copre tutti i requisiti) e **leggibile**

Esempio: per un database scolastico non interessa modellare l'altezza degli studenti, ma è rilevante modellare la loro classe di appartenenza e le materie frequentate.

Dalla realtà allo schema: i tre livelli

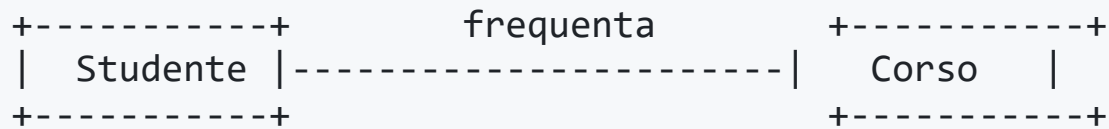
Come visto per l'architettura del DBMS, anche la progettazione segue **tre livelli di astrazione**, coerenti con i tre livelli ANSI-SPARC:

Livello	Cosa rappresenta	Strumento
Concettuale	La realtà in modo astratto, indipendente dal DBMS	Schema E/R
Logico	La struttura dei dati secondo un modello specifico, pronta per manipolazione e interrogazione	Schema relazionale (tabelle, chiavi)
Fisico	Come i dati sono effettivamente memorizzati	File, indici, strutture di accesso

Il livello **logico** è quello su cui si eseguono operazioni di **manipolazione (DML)** e **interrogazione (QL)** tramite SQL; il livello **concettuale** serve solo per l'analisi e la progettazione.

Schema/diagramma ER

Lo **schema ER** (o **diagramma ER**) è la rappresentazione grafica del modello concettuale: un insieme di **entità**, **associazioni** e **attributi**, collegati da simboli grafici standardizzati.



- Le **entità** si rappresentano con **rettangoli**
- Le **associazioni** si rappresentano linee che uniscono due o più entità (oppure con **rombi**)
- Gli **attributi** si rappresentano con **ellissi** (o, in notazioni semplificate, elencati in un riquadro)

Lo schema ER è il documento di riferimento condiviso tra progettista e committente durante l'analisi.

Le entità

Una **entità** un oggetto (concreto o astratto) che ha un significato anche quando viene considerato in modo isolato ed è di interesse per la realtà che si vuole modellare.

- Si rappresenta con un **rettangolo**, etichettato con un nome **singolare** e in **maiuscolo** (convenzione)
- Ogni **occorrenza** (istanza) dell'entità corrisponde a un oggetto specifico del mondo reale

```
+-----+  
|  STUDENTE  |  
+-----+
```

Esempio: l'entità `STUDENTE` rappresenta la classe di tutti gli studenti; "Mario Rossi" è una singola **occorrenza** di quell'entità.

Il dominio di un attributo

Il **dominio** di un attributo è l'**insieme dei valori ammissibili** che quell'attributo può assumere.

- Definisce sia il **tipo di dato** (testuale, numerico, data, booleano...) sia, eventualmente, un **intervallo o insieme di valori validi**
- È già una scelta rilevante a livello **concettuale**, anche se verrà formalizzata tecnicamente nel livello logico/fisico (es. con `CHECK` in SQL)

Attributo	Dominio
Sesso	{M, F}
Voto	Interi tra 0 e 10
DataNascita	Date valide ($\leq$ data odierna)
Email	Stringhe con formato <code>nome@dominio</code>

Esempio d'esame: definire il dominio di `voto` come "interi compresi tra 0 e 10" evita, già in fase di analisi, ambiguità su cosa sia un valore accettabile.

Gli attributi: caratteristiche

Gli **attributi** descrivono le proprietà di un'entità (o di un'associazione). Ogni attributo ha:

- **Formato (tipo di dato)**: es. testo, numero, data — coerente col suo **dominio**
- **Dimensione**: la lunghezza massima consentita (es. `VARCHAR(30)`)
- **Opzionalità**: se l'attributo è **obbligatorio** o **facoltativo**
- **Valori nulli (NULL)**: rappresentano l'**assenza di informazione**, ammessi solo per attributi opzionali

Attributo	Formato	Dimensione	Opzionale?
Cognome	Testo	30 caratteri	No
Email	Testo	50 caratteri	Sì (può essere NULL)
DataNascita	Data	—	No

Attenzione: `NULL` non equivale a zero o stringa vuota: significa "valore sconosciuto o non applicabile".

Attributi semplici, composti e multivalore

- **Attributo semplice (atomico)**: non ulteriormente scomponibile (es. `CodiceFiscale`)
- **Attributo composto**: risulta dall'unione di più sotto-attributi (es. `Indirizzo` = `Via` + `Città` + `CAP`)
- **Attributo multivalore**: può assumere **più valori contemporaneamente** per la stessa occorrenza (es. `NumeroTelefono` , se una persona può averne diversi) — si rappresenta con un **doppio contorno** (ellisse o riquadro doppio)

Esempio: l'entità `PERSONA` può avere l'attributo composto `Indirizzo` (scomposto in `Via` , `Città` , `CAP`) e l'attributo multivalore `Telefono` (una persona può avere più numeri).

Attributi derivati

Un **attributo derivato** è un attributo il cui valore **può essere calcolato** a partire da altri attributi (o da altre informazioni presenti nel database), invece di essere memorizzato direttamente.

- Si rappresenta con un'**ellisse tratteggiata** (contorno a puntini)
- Non introduce nuova informazione, ma **ridondanza calcolabile**: si può scegliere se memorizzarlo fisicamente (per efficienza) o calcolarlo ogni volta (per evitare inconsistenze)

```
DataNascita — STUDENTE - - - - (Età)
                        ^
                    attributo derivato
                    (calcolato da DataNascita)
```

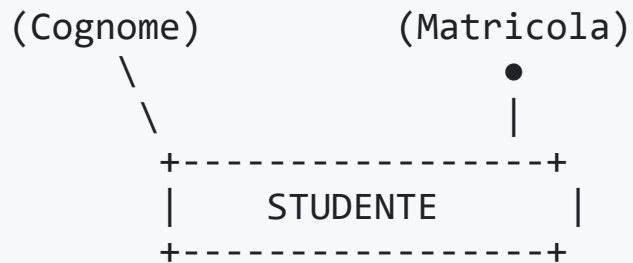
Esempio classico: l'attributo `Età` è derivato da `DataNascita`: memorizzarlo entrambi crea il rischio di **inconsistenza** se non si aggiornano insieme (violazione della consistenza vista in precedenza).

Rappresentazione grafica degli attributi

Esistono **due notazioni comuni** per rappresentare gli attributi di un'entità nello schema ER:

1) Notazione classica (Chen) — l'attributo è un'**ellisse** collegata all'entità da una linea che termina con un **piccolo cerchio**:

- **cerchio pieno (●)** → l'attributo fa parte della **chiave primaria**
- **cerchio vuoto (○)** → attributo non chiave



2) Notazione semplificata (a rettangolo) — gli attributi sono elencati in un **riquadro** sotto (o dentro) il nome dell'entità, con la chiave primaria **sottolineata**: è la notazione più usata nei libri di testo e nei temi d'esame per la sua compattezza.

Chiave primaria (PK)

La **chiave primaria (Primary Key, PK)** è l'attributo (o insieme minimo di attributi) che permette di **identificare univocamente** ogni occorrenza di un'entità.

- Deve rispettare due proprietà: **unicità** (non due occorrenze con lo stesso valore) e **non nullità** (non può mancare)
- Nello schema ER si evidenzia **sottolineando** l'attributo (notazione a rettangolo) o con il **cerchio pieno** (notazione classica)

+	-----	+
	STUDENTE	
+	-----	+
	Matricola (PK)	
	Cognome	
	Nome	
+	-----	+

<- sottolineato nello schema

Esempio: in `STUDENTE`, l'attributo `Matricola` è scelto come chiave primaria perché univoco per ogni studente, a differenza di `Cognome` (potrebbero esserci omonimi).

Chiave primaria parziale (PPK)

La **chiave primaria parziale (Partial Primary Key, PPK)** è l'attributo (o insieme di attributi) che identifica univocamente le occorrenze di un'**entità debole**, ma solo **all'interno del contesto** dell'entità forte a cui è collegata: da sola non è sufficiente a livello di intero database.

	Chiave primaria (PK)	Chiave primaria parziale (PPK)
Appartiene a	Entità forte	Entità debole
Identifica univocamente...	...su tutto il database	...solo insieme alla PK dell'entità forte collegata
Rappresentazione	Sottolineatura continua	Sottolineatura tratteggiata

Esempio: RIGA_FATTURA ha NumRiga come **PPK**: il NumRiga "1" esiste in molte fatture diverse; la vera chiave completa è (NumFattura, NumRiga), dove NumFattura è la PK di FATTURA.

Entità forti ed entità deboli

- Un'**entità forte** possiede una chiave primaria **propria (PK)**, che la identifica autonomamente, indipendentemente da altre entità
- Un'**entità debole non ha** una PK propria: la sua identificazione dipende dalla combinazione della propria **chiave primaria parziale (PPK)** con la PK di un'**entità forte** correlata, tramite un'associazione detta **identificante**

```
+-----+ 1          N +-----+
| FATTURA |-----| RIGA_FATTURA |
+-----+ (forte)    +-----+ (debole)
   NumFattura (PK)      NumRiga (PPK)
                        + NumFattura (ereditata)
```

Esempio: RIGA_FATTURA è entità debole: il NumRiga da solo non basta a identificarla univocamente nel database, serve la combinazione con NumFattura della fattura a cui appartiene.

Associazioni tra entità

Un'**associazione** rappresenta un legame logico tra due (o più) entità.

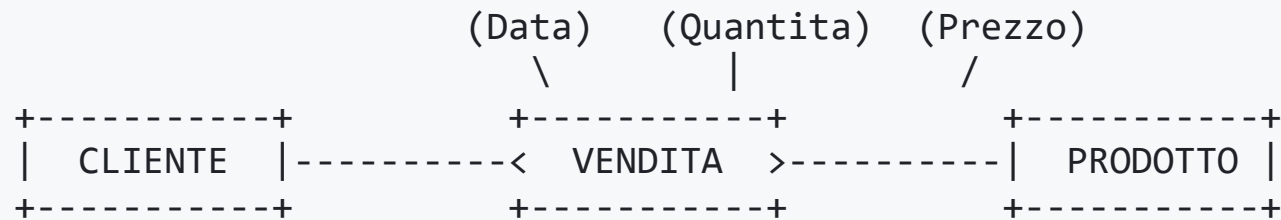
- Si rappresenta con un **rombo**, collegato alle entità coinvolte tramite linee
- Ha un **nome** che ne descrive il significato, solitamente un verbo (es. frequenta , insegna , appartiene a)

```
+-----+           +-----+           +-----+
| STUDENTE |-----< FREQUENTA >-----| CORSO |
+-----+           +-----+           +-----+
```

Un'associazione, come un'entità, può avere una propria **molteplicità di partecipazione** per ciascuna entità coinvolta (vedi cardinalità nelle prossime slide).

Gli attributi delle associazioni

Anche le **associazioni**, non solo le entità, possono avere **propri attributi**: si usano quando l'informazione non appartiene a nessuna delle due entità singolarmente, ma nasce dalla **loro combinazione**.



Esempio: l'associazione VENDITA tra CLIENTE e PRODOTTO ha attributi Data , Quantità , Prezzo : questi valori non hanno senso da soli per il cliente o per il prodotto, ma solo riferiti a **una specifica vendita** (combinazione dei due).

Regola pratica: se un dato dipende **da entrambe** le entità coinvolte (e cambia per ogni combinazione), va posto come attributo dell'**associazione**, non di una singola entità.

Il grado di un'associazione

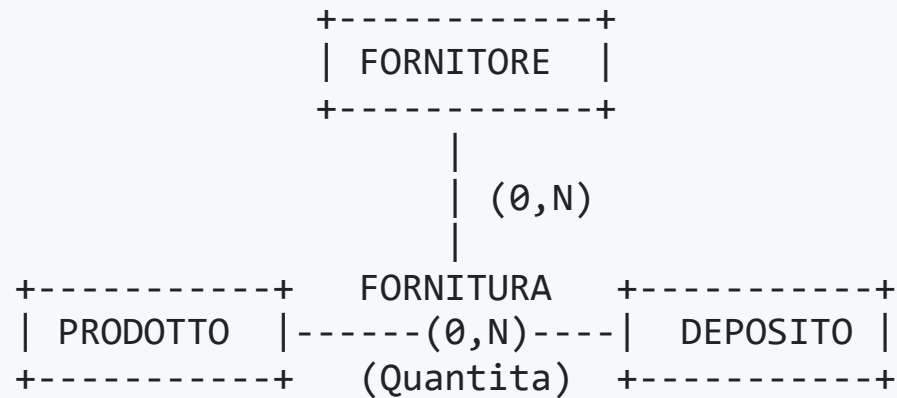
Il **grado** di un'associazione è il **numero di entità coinvolte** nell'associazione stessa:

- **Associazione binaria** (grado 2): coinvolge **due** entità — è il caso più comune (es. `STUDENTE – FREQUENTA – CORSO`)
- **Associazione ternaria** (grado 3): coinvolge **tre** entità contemporaneamente
- **Associazione n-aria**: coinvolge **n** entità (generalizzazione, usata raramente)
- **Associazione ricorsiva (o unaria/riflessiva)**: collega un'entità **con se stessa**

Attenzione: un'associazione ternaria **non equivale** a tre associazioni binarie separate: rappresenta un legame che esiste **solo quando tutte e tre le entità partecipano insieme** alla stessa occorrenza.

Esempio di associazione ternaria

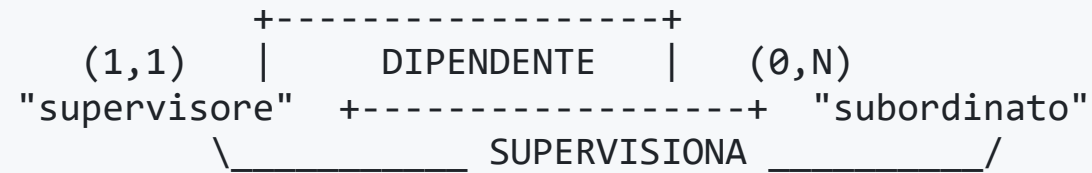
Realtà: un FORNITORE fornisce un PRODOTTO a un DEPOSITO , con una certa Quantità .



Non si può scomporre in tre associazioni binarie (Fornitore-Prodotto , Prodotto-Deposito , Fornitore-Deposito) senza **perdere informazione**: la tripla esatta (quale fornitore, quale prodotto, in quale deposito, quanto) esiste solo come associazione ternaria unica.

Esempio di associazione ricorsiva

Un'**associazione ricorsiva** collega occorrenze della **stessa entità** tra loro. Ai due "estremi" dell'associazione si assegnano **ruoli** diversi per distinguere le due partecipazioni.



Esempio: l'entità **DIPENDENTE** è collegata a se stessa dall'associazione **SUPERVISIONA** : un dipendente (nel ruolo di supervisore) può supervisionare da 0 a N altri dipendenti; ogni dipendente (nel ruolo di subordinato) ha esattamente 1 supervisore.

Molteplicità e cardinalità

La **cardinalità** (o molteplicità) di un'associazione indica **quante occorrenze** di un'entità possono essere associate a occorrenze dell'altra entità.

Si esprime con una coppia di valori (**minima, massima**) per ciascuna entità coinvolta:

- **Cardinalità minima:** 0 (partecipazione **opzionale**) oppure 1 (partecipazione **obbligatoria**)
- **Cardinalità massima:** 1 (al più una occorrenza) oppure N (molte occorrenze)

Le combinazioni tipiche sono: **(0,1)**, **(1,1)**, **(0,N)**, **(1,N)**

Esempio: in STUDENTE (1,1) – FREQUENTA – (0,N) CORSO : ogni studente frequenta esattamente 1 corso alla volta (min 1, max 1); ogni corso può essere frequentato da zero o più studenti (min 0, max N).

Cardinalità: altri esempi pratici

Associazione	Entità A (card.)	Entità B (card.)	Interpretazione
PERSONA – POSSIEDE – AUTOMOBILE	(0,N)	(1,1)	una persona possiede 0 o più auto; un'auto appartiene a 1 sola persona
PAZIENTE – PRENOTA – VISITA	(0,N)	(1,1)	un paziente prenota 0 o più visite; una visita è di 1 solo paziente
DOCENTE – COORDINA – CLASSE	(0,1)	(1,1)	un docente coordina al più 1 classe (facoltativo); una classe ha esattamente 1 coordinatore
AUTORE – SCRIVE – LIBRO	(1,N)	(1,N)	un autore scrive 1 o più libri; un libro può avere 1 o più autori (N:N)

Osservare **sempre entrambe** le cardinalità (minima e massima) evita errori comuni nei temi d'esame, dove spesso si chiede di giustificare la scelta con un esempio concreto.

I tre tipi di associazione

In base alle cardinalità **massime**, un'associazione binaria tra due entità può essere di tre tipi:

Tipo	Cardinalità massime	Esempio
Uno a uno (1:1)	1 — 1	PERSONA possiede CARTA_IDENTITA
Uno a molti (1:N)	1 — N	CLASSE è composta da STUDENTE
Molti a molti (N:N)	N — N	STUDENTE frequenta CORSO

Ulteriori esempi:

- **1:1** → NAZIONE — HA_CAPITALE — CITTA
- **1:N** → REPARTO — IMPIEGA — DIPENDENTE
- **N:N** → ATTORE — RECITA_IN — FILM

Attenzione per l'esame: un'associazione **N:N** a livello logico (relazionale) **non può essere tradotta direttamente**: richiede una tabella ponte (entità associativa) con le chiavi esterne di entrambe le entità coinvolte.

Regole di lettura di un'associazione

Per leggere correttamente le cardinalità in uno schema ER, si applica questa regola:

"Una occorrenza di **[Entità A]** è associata a **[cardinalità minima, cardinalità massima]** occorrenze di **[Entità B]**"

Esempio pratico:

DOCENTE (1,N) ----- INSEGNA ----- (1,1) MATERIA

- Letto da **DOCENTE** verso **MATERIA** : *"un docente insegna da 1 a N materie"*
- Letto da **MATERIA** verso **DOCENTE** : *"una materia è insegnata da esattamente 1 docente"*

Trucco mnemonico: la cardinalità scritta **vicino a un'entità** descrive **quante occorrenze di quell'entità** partecipano all'associazione, viste dal punto di vista dell'**altra** entità.

Simboli grafici: notazione classica (Chen)

Nella notazione classica del modello E/R (Chen):

- **Rettangolo** → entità
- **Rettangolo a doppio contorno** → entità debole
- **Rombo** → associazione
- **Ellisse** → attributo, collegata con una linea che termina con un **cerchio pieno (PK)** o **vuoto (non chiave)**
- **Ellisse tratteggiata** → attributo derivato
- **Ellisse a doppio contorno** → attributo multivalore
- **Linea singola** → partecipazione opzionale/parziale
- **Linea doppia** → partecipazione totale/obbligatoria
- Le cardinalità si scrivono come coppie **(min, max)** accanto alla linea di collegamento

```
STUDENTE ===(1,1)=== FREQUENTA --- (0,N) --- CORSO  
      (linea doppia = obbligatoria)  (linea singola = opzionale)
```

Simboli grafici: notazione crow's foot (zampa di gallina)

La notazione **crow's foot** ("zampa di gallina") è molto usata negli strumenti CASE moderni e nei diagrammi UML/ERD (è anche la notazione prodotta di default da strumenti come **draw.io**):

- **Tratto singolo perpendicolare** ($-|-$) → cardinalità massima **1**
- **Zampa di gallina** ($-<$) → cardinalità massima **N** (molti)
- **Cerchio** ($\circ$) vicino all'entità → cardinalità minima **0** (opzionale)
- **Tratto singolo** vicino all'entità → cardinalità minima **1** (obbligatoria)

```
STUDENTE |-----< FREQUENTA >-----0 CORSO
          (1,N: molti studenti)      (0,N: zero o più corsi)
```

Combinazioni frequenti nei testi d'esame: $\circ<$ = (0,N); $|<$ = (1,N); $\circ|$ = (0,1); $||$ = (1,1).

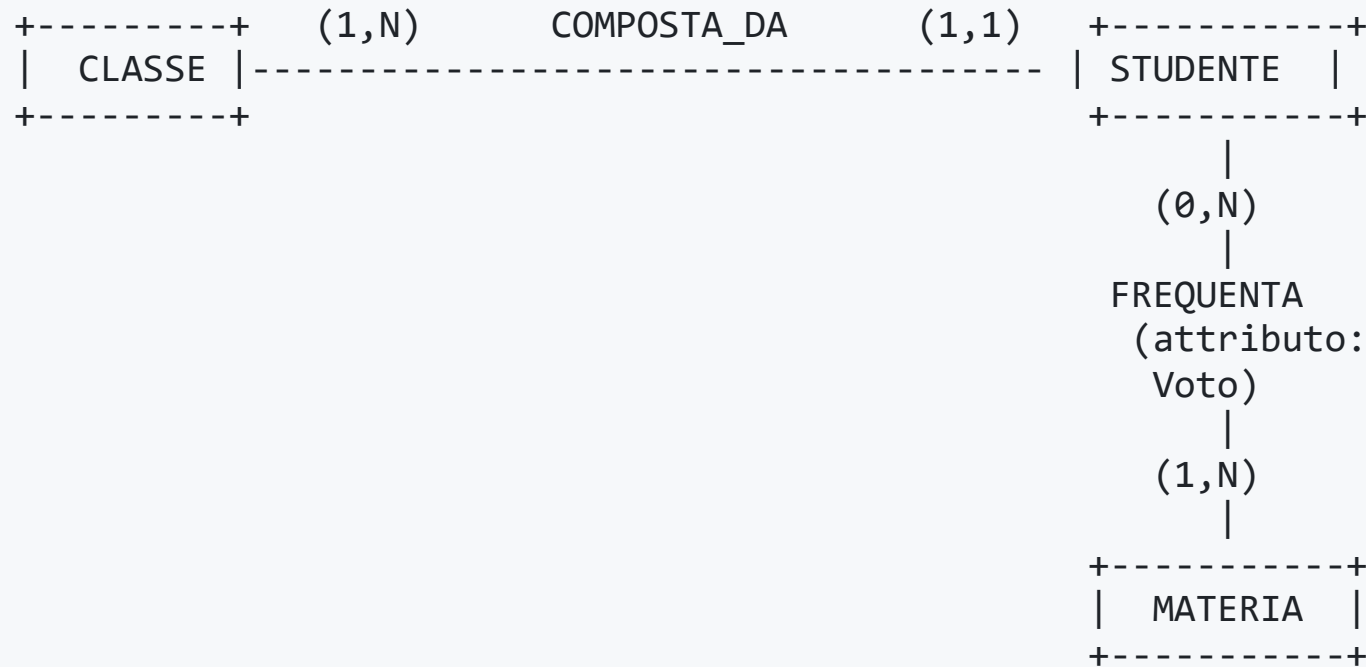
Confronto tra le due notazioni

Cardinalità	Notazione Chen	Notazione crow's foot
(1,1) obbligatoria, singola	linea doppia + (1,1)	
(0,1) opzionale, singola	linea singola + (0,1)	o
(1,N) obbligatoria, multipla	linea doppia + (1,N)	<
(0,N) opzionale, multipla	linea singola + (0,N)	o<

Entrambe le notazioni esprimono **gli stessi concetti**: nei compiti/simulazioni è importante saper riconoscere e usare **almeno una** delle due in modo coerente, dichiarando la legenda se necessario.

Esempio completo di schema ER

Realtà: una scuola gestisce classi, studenti e materie.



- CLASSE (1,N) – COMPOSTA_DA – (1,1) STUDENTE : ogni classe ha molti studenti, ogni studente appartiene a esattamente una classe
- STUDENTE (0,N) – FREQUENTA – (1,N) MATERIA : associazione N:N con attributo Voto

L'importanza dell'analisi del problema

Una progettazione concettuale efficace **dipende innanzitutto da un'analisi accurata dei requisiti**, prima ancora di disegnare lo schema.

- Individuare **tutti gli oggetti rilevanti** (entità) e le loro **proprietà** (attributi, con relativo dominio)
- Comprendere **le regole del dominio applicativo** (es. un docente può insegnare più materie? uno studente può essere in più classi?)
- Chiarire con il committente i **casi ambigui** prima di formalizzare cardinalità e vincoli
- Un errore nell'analisi si **ripercuote** su schema logico, schema fisico e, in ultima analisi, sul software applicativo

Da ricordare per l'esame: la qualità del **modello dei dati** dipende dalla qualità dell'**analisi del problema**: un buon progettista dedica tempo a comprendere la realtà prima di modellarla.

Mappa concettuale riassuntiva

ANALISI DEL PROBLEMA

|

v

PROGETTAZIONE CONCETTUALE (modello E/R, UML)

| -- Entita' (forti/deboli) --> PK / PPK

| -- Attributi --> dominio, formato, dimensione, opzionalita', NULL

| --> semplici, composti, multivalore, derivati

| -- Associazioni --> attributi propri, grado (binaria/ternaria/ricorsiva)

| --> molteplicita'/cardinalita' (0,1 / 1,1 / 0,N / 1,N)

| --> tipi: 1:1, 1:N, N:N

| -- Notazioni grafiche: Chen (rombi, ellissi) / crow's foot (zampa di gallina)

|

v

SCHEMA LOGICO (modello relazionale) --> tabelle, chiavi, vincoli

|

v

SCHEMA FISICO --> file, indici, DBMS specifico

Domande guida per il ripasso

- Perché la progettazione concettuale deve essere indipendente dal DBMS?
- Cos'è il dominio di un attributo? Fai un esempio
- Differenza tra chiave primaria (PK) e chiave primaria parziale (PPK): quando si usa quest'ultima?
- Cosa sono gli attributi derivati e come si rappresentano graficamente?
- In che modo le associazioni possono avere attributi propri? Fai un esempio
- Cos'è il grado di un'associazione? Spiega la differenza tra binaria, ternaria e ricorsiva con un esempio ciascuna
- Come si legge la cardinalità di un'associazione? Fai un esempio con (1,N) e (0,N)
- Perché un'associazione N:N richiede un trattamento particolare nel passaggio al modello logico?
- Disegna lo schema ER di una realtà a scelta (es. Ordini-Prodotti-Clienti) indicando entità, attributi, chiavi e cardinalità

Fine

Domande?